

TraceForge v2 — Complete Demo & Usage Guide

Purpose: Step-by-step guide for running a live demonstration of TraceForge v2 using real evidence collected from your own Linux system.

Target OS: Ubuntu 22.04 LTS

Time required: 30 minutes to prepare, 7-8 minutes to record

Before You Start — Setup Check

Every time you open a terminal to use TraceForge, run these two commands first. Do not skip this. If the venv is not active, nothing will work.

```
cd ~/traceforge
source venv/bin/activate
```

Verify the tool is ready:

```
python -m traceforge.cli --help
```

You should see the full list of subcommands. If you get a `ModuleNotFoundError`, the venv is not active.

Also verify your sample directories exist:

```
ls ~/traceforge/samples/
```

Expected output:

```
disk logs memory network
```

If any folder is missing:

```
mkdir -p ~/traceforge/samples/{logs,network,memory,disk}
```

Part 1 — Collect Real Evidence From Your Own Linux System

This is what makes the demo real. You are not using fake sample files. You are collecting actual forensic evidence from your own running machine.

Step 1.1 — Collect auth.log (real login history)

Your system's auth.log contains real failed logins, successful logins, sudo commands, and service events. This is genuine forensic data.

```
sudo cp /var/log/auth.log ~/traceforge/samples/logs/auth.log
sudo chmod 644 ~/traceforge/samples/logs/auth.log
```

Verify it has content:

```
wc -l ~/traceforge/samples/logs/auth.log
```

You should see at least a few hundred lines. If auth.log is empty or missing, use syslog instead:

```
sudo cp /var/log/syslog ~/traceforge/samples/logs/auth.log
sudo chmod 644 ~/traceforge/samples/logs/auth.log
```

Step 1.2 — Capture live network traffic

This captures real network traffic from your machine. Run this command, then browse the web or ping something to generate traffic, then wait for it to finish.

Terminal 1 — Start the capture:

```
sudo tcpdump -i any -w ~/traceforge/samples/network/live_capture.pcap -c 200
```

Terminal 2 — Generate real traffic while capturing:

```
ping google.com -c 10
curl https://example.com
curl https://github.com
```

Wait for tcpdump to print `200 packets captured` and exit. Then verify:

```
ls -lh ~/traceforge/samples/network/live_capture.pcap
```

The file should be at least a few KB in size.

Why -c 200? It captures exactly 200 packets then stops automatically. No need to Ctrl+C.

Step 1.3 — Prepare the disk evidence

The disk module works on directories — no raw disk image needed. Your home directory is real forensic data: real file timestamps, real file metadata, real access times.

No command needed here. You will point the disk module at `/home/theconsoler` during analysis.

Step 1.4 — Optional — Memory sample

If you want to demonstrate the memory module, capture a kernel memory sample. This is optional for the demo since it requires root and produces a large file.

```
sudo mkdir -p ~/traceforge/samples/memory
sudo dd if=/proc/kcore of=~/traceforge/samples/memory/system.raw bs=1M count=256 2>/dev/null
sudo chmod 644 ~/traceforge/samples/memory/system.raw
```

Note: Volatility3 needs a matching symbol table to parse the memory image. For a demo without memory analysis, skip this and focus on logs + network + disk which all work without additional setup.

Part 2 — The Demo Script

Follow these scenes in order. Each scene builds on the previous one. Do not skip scenes.

Scene 1 — Launch the interactive tool

```
cd ~/traceforge
source venv/bin/activate
python -m traceforge
```

What to show the audience:

- The ASCII banner at the top
- The live stats panels: Total Cases, Total Artifacts, Today's date — these are pulled live from the database
- The Recent Cases table showing your previous investigations
- The colour-coded menu with all seven options

What to say: *"This is the TraceForge launcher. Every number you see here is live data from the investigation database. 49 cases, 113 artifacts — all real work from previous sessions. The menu covers every workflow from a single screen."*

Scene 2 — Create a new investigation case

Select option **1** from the menu and fill in:

```
Case ID       : DEMO-2026-002
Case name     : Live Anamoly Detection
Analyst name  : theconsoler
Notes        : Live demonstration investigation
```

Or directly via CLI:

```
python -m traceforge.cli case new \
  --id DEMO-2026-002 \
  --name "Linux System Anomaly Investigation" \
  --analyst theconsoler \
  --notes "Live demonstration investigation"
```

What to show:

- The case created confirmation with ID, name, analyst, and timestamp
- Return to the launcher and notice the Total Cases counter has incremented

What to say: *"Every investigation starts with a case. The case ID, analyst name, and creation timestamp are all recorded. This is how professional incident response works — every action is attributed to an analyst and timestamped."*

Scene 3 — Analyze the auth log

Select option **2** and fill in:

```
Case ID           : DEMO-2026-002
Module            : logs
Evidence file path : samples/logs/auth.log
Analyst name      : theconsoler
Host identifier    : ubuntu-machine
```

Or via CLI:

```
python -m traceforge.cli analyze \
  --case DEMO-2026-002 \
  --module logs \
  --file samples/logs/auth.log \
  --analyst theconsoler \
  --host ubuntu-machine
```

What to show and point out:

```
Computing SHA-256 for: auth.log
SHA-256: 8f62d9687205ffc469a6ffed7fbe3fadb... | Size: 58,299 bytes
Evidence logged: samples/logs/auth.log | Case: DEMO-2026-002
```

- The SHA-256 hash is computed on the raw file BEFORE analysis begins
- The evidence is recorded in the append-only ledger
- The artifacts found — failed logins, successful logins, sudo commands, brute force detections

What to say: *"Before the module touches the file, TraceForge hashes it. This SHA-256 is recorded in the evidence ledger and cannot be modified. If this file is tampered with after ingestion, the hash will not match — that is chain of custody. Now watch what the log module finds from your real system."*

Point out specifically if it finds:

- Failed login attempts from any IP
 - Brute force detection (5+ failed attempts from same IP)
 - Sudo commands showing privilege escalation
 - Successful logins
-

Scene 4 — Analyze the network capture

Select option **2** again and fill in:

```
Case ID           : DEMO-2026-002
Module            : network
Evidence file path : samples/network/live_capture.pcap
Analyst name      : theconsoler
Host identifier    : 192.168.223.140
```

Or via CLI:

```
python -m traceforge.cli analyze \
  --case DEMO-2026-002 \
  --module network \
  --file samples/network/live_capture.pcap \
  --analyst theconsoler \
  --host 192.168.223.140
```

What to show and point out:

- DNS queries extracted — real domains your machine looked up
- HTTP requests reconstructed — method, host, URI
- TCP flows — source IP, destination IP, ports, bytes transferred
- Any connection to suspicious ports (4444, 1337, 31337) gets flagged as `suspicious_connection`

What to say: *"This is your machine's real network activity from the last few minutes. Every DNS query, every TCP connection, every HTTP request is now an artifact in the investigation database. The same case, same analyst, same chain of custody as the log analysis."*

Scene 5 — Analyze the disk (home directory)

Select option **2** again and fill in:

```
Case ID           : DEMO-2026-002
Module            : disk
Evidence file path : /home/theconsoler
Analyst name      : theconsoler
Host identifier    : ubuntu-machine
```

Or via CLI:

```
python -m traceforge.cli analyze \
  --case DEMO-2026-002 \
  --module disk \
  --file /home/theconsoler \
  --analyst theconsoler \
  --host ubuntu-machine
```

What to show:

- Every file in your home directory becomes an artifact
- Three timestamps per file: created, modified, accessed
- Deleted file markers if any are found

What to say: *"The disk module builds a complete file timeline from a disk image or a live directory. Every file entry includes three timestamps — created, modified, and last accessed. In a real investigation, this tells you exactly what files were touched and when."*

Scene 6 — Run the correlation engine

This is the most important scene in the demo.

Select option **3** from the menu, or via CLI:

```
python -m traceforge.cli correlate --case DEMO-2026-002
```

What to show:

```
Cross-Module Correlation Engine
Case: DEMO-2026-002 | Window: 30s
```

```
Correlation complete
```

```
Total links      : X  
High confidence  : X  
Medium confidence: X  
Low confidence   : X
```

```
85% LOGS [failed_login] and NETWORK [tcp_flow] share IP address: X.X.X.X
```

```
90% LOGS [failed_login] and NETWORK [suspicious_connection] occurred within 5.2s
```

What to say: *"This is what makes TraceForge different from five separate tools. The correlation engine just automatically found that the same IP address appears in both the log analysis and the network capture. That is a cross-source finding you would normally spend an hour finding manually. High confidence means the evidence is strong. This is your attack timeline starting to emerge."*

If zero correlations appear: This means the same IPs do not appear in both your auth.log and your PCAP, or the timestamps are too far apart. This is normal for a clean system. You can still show the correlation engine ran and explain what it would find in a real incident. For the demo, the important thing is showing the mechanism.

Scene 7 — Generate the full report

Select option **4** from the menu, or via CLI:

```
python -m traceforge.cli report --case DEMO-2026-002 --format all
```

What to show:

```
Generating Case Report
```

```
Case      : DEMO-2026-002  
Formats   : ['json', 'html', 'pdf']
```

```
Report generated successfully
```

```
JSON      : /home/theconsoler/traceforge/reports/report_DEMO-2026-002_...json  
HTML      : /home/theconsoler/traceforge/reports/report_DEMO-2026-002_...html  
PDF       : /home/theconsoler/traceforge/reports/report_DEMO-2026-002_...pdf
```

Open the HTML report:


```
xdg-open ~/traceforge/reports/report_DEMO-2026-002_*.html
```

Walk through the HTML report with the audience:

1. The header — case ID, analyst name, generated timestamp
2. The summary stats panel — total artifacts, correlations, critical findings, timeline events
3. The artifacts by module table — shows how many artifacts each module found
4. The evidence ledger section — every evidence file with its full SHA-256 hash, size, and recorded timestamp
5. The correlation findings section — every cross-module link with confidence score and description
6. The IOC timeline — all artifacts from all modules sorted chronologically with severity badges

What to say: *"One command, three output formats. The JSON is machine-readable — you can feed it into a SIEM or another tool. The HTML is for investigators. The PDF is for formal case documentation. Notice the evidence ledger — every file you analyzed is here with its SHA-256 hash. This report is court-ready documentation."*

Scene 8 — Open the web dashboard

Open a second terminal window:

```
cd ~/traceforge
source venv/bin/activate
python -m dashboard.app
```

Open browser at `http://127.0.0.1:5000`

Walk through each page:

Home page — Case List:

- Show all cases listed with artifact counts and module badges
- Point out DEMO-2026-002 with its artifact count

- Click View to enter the case

Case Overview page:

- Show the stats panel — Total Artifacts, Correlations, Critical, High, Medium, Timeline Events
- Point out the Artifacts by Module table
- Show the Evidence Ledger with SHA-256 hashes
- Point out the export buttons at the top — Export HTML, Export PDF, Export JSON

Timeline page (click Timeline button):

- Show the full IOC timeline sorted chronologically
- Use the filter buttons to show only LOGS — notice only log artifacts
- Switch to NETWORK — notice only network artifacts
- Switch to ALL to show everything
- Filter by HIGH severity to show only high-severity findings

Correlations page (click Correlations button):

- Show each correlation with confidence percentage
- Point out the link type column — timestamp_proximity vs ip_match vs host_match
- Point out the module badges showing which two modules are connected

What to say: *"The dashboard reads from the exact same database the CLI writes to. There is no sync, no API call, no duplication. Run your investigation from the terminal, present your findings in the browser. The timeline filter lets you isolate specific module findings or severity levels — exactly what you need when briefing a team."*

Scene 9 — Show the GitHub repository

Open browser and go to <https://github.com/theconsolet/TRACEFORGE-V2>

Walk through:

- The README with the ASCII banner
- The project structure

- The build phases documented in the docs folder
- The 36 tests

What to say: *"TraceForge v2 is fully open source. The entire build is documented phase by phase. Anyone can clone this, follow the setup guide, and run it on Ubuntu 22.04. Four build phases, 36 automated tests, all passing."*

Part 3 — What to Say at Each Key Moment

Moment	What to say
Opening	"Analysts normally use 4-5 separate tools for a single investigation. TraceForge brings all of that under one roof."
SHA-256 hash appearing	"That hash is computed on the raw file before analysis begins. It is written to an append-only ledger. This is chain of custody."
Brute force detection	"The log module automatically detected a brute force attack — 5 or more failed logins from the same IP. No manual grep required."
Network artifacts	"These are real DNS queries and TCP connections your machine just made. Every connection is now a forensic artifact."
Correlation result	"The engine linked a log artifact and a network artifact because they share the same IP address. That is a finding that normally takes 20 minutes to find manually."
Report generation	"One command produces JSON, HTML, and PDF. The HTML report is investigator-ready, the PDF is documentation-ready."
Dashboard timeline filter	"You can filter the entire timeline by module or severity in one click. In a real incident with thousands of artifacts, this saves hours."
Closing	"This tool was rebuilt from scratch from a 7th semester skeleton that never shipped. v2 is the real thing — functional, tested, documented, and open source."

Part 4 — Demo Video Structure

Timestamp	Scene	What to Show
0:00 — 0:30	Introduction	Explain the problem TraceForge solves. Show the tool name and GitHub link.
0:30 — 1:00	Scene 1	Launch the interactive launcher. Show banner, live stats, recent cases, menu.

Timestamp	Scene	What to Show
1:00 — 2:00	Scene 2	Create DEMO-2026-002. Explain case management and analyst attribution.
2:00 — 3:30	Scene 3	Analyze auth.log. Show SHA-256 hash, evidence ledger, real findings.
3:30 — 4:30	Scene 4	Analyze live_capture.pcap. Show DNS queries, TCP flows, suspicious ports.
4:30 — 5:00	Scene 5	Analyze home directory. Show file timeline and timestamps.
5:00 — 5:45	Scene 6	Run correlation engine. Show cross-module links and confidence scores.
5:45 — 6:30	Scene 7	Generate report. Open HTML in browser. Walk through evidence ledger and timeline.
6:30 — 7:30	Scene 8	Open dashboard. Walk through timeline filter and correlations page.
7:30 — 8:00	Scene 9	Show GitHub repo. Close with purpose statement.

Part 5 — Pre-Recording Checklist

Run through this before pressing record:

- ✓ ~~Terminal font size increased to at least 14pt for readability~~
- ✓ ~~Screen resolution set to 1920×1080~~
- ✓ ~~auth.log copied from /var/log/auth.log and has content~~
- ✓ ~~live_capture.pcap exists and is not empty~~
- ✓ ~~DEMO-2026-002 case does NOT exist yet (create it fresh on camera)~~
- ✓ ~~Dashboard is not running yet (start it on camera)~~
- ✓ ~~Browser has no tabs open except a blank one~~
- ✓ ~~GitHub repo is bookmarked for Scene 9~~
- ✓ ~~Full dry run completed once without recording~~
- ✓ ~~Two terminal windows ready — one for CLI, one for dashboard~~

Part 6 — Common Issues During Demo

"Case does not exist"

You tried to analyze evidence before creating the case. Always run Scene 2 first.

"PCAP file not found"

Run the tcpdump capture command from Part 1 Step 1.2 before Scene 4.

"Zero correlations found"

Normal on a clean system with no active attacks. Explain the mechanism to the audience and show what correlation results look like from a previous case. You can also use CASE-001 or CASE-2026-003 which already have correlations from earlier sessions.

"Dashboard shows no cases"

The dashboard reads from `~/traceforge/data/traceforge.db`. If you ran analysis in a different directory, the database path may be different. Always run everything from `~/traceforge`.

"Module ran but zero artifacts found"

Check the file path is correct and the file has content. For auth.log, verify it is not empty with `wc -l samples/logs/auth.log`.

Part 7 — Using TraceForge for Real Investigations on Your Own System

Beyond the demo, here is how to use TraceForge for actual anomaly detection on your Linux system.

Weekly health check — run this every week:

```
cd ~/traceforge
source venv/bin/activate

# Create a dated case
python -m traceforge.cli case new \
  --id "HEALTH-$(date +%Y%m%d)" \
  --name "Weekly System Health Check" \
  --analyst theconsoler

# Analyze latest auth log
sudo cp /var/log/auth.log samples/logs/auth_$(date +%Y%m%d).log
```

```

sudo chmod 644 samples/logs/auth_$(date +%Y%m%d).log
python -m traceforge.cli analyze \
  --case "HEALTH-$(date +%Y%m%d)" \
  --module logs \
  --file samples/logs/auth_$(date +%Y%m%d).log \
  --analyst theconsoler \
  --host $(hostname)

# Correlate and generate report
python -m traceforge.cli correlate --case "HEALTH-$(date +%Y%m%d)"
python -m traceforge.cli report --case "HEALTH-$(date +%Y%m%d)" --format html

```

What to look for in the report:

- Any IP with 5+ failed logins — active brute force attempt
- Any successful login from an unfamiliar IP — potential compromise
- Any sudo command you did not run yourself — privilege escalation
- Any outbound connection to port 4444, 1337, or 31337 — reverse shell indicator
- Any DNS query to an unfamiliar domain — potential C2 communication

When you find something suspicious:

1. Note the IP address or domain
2. Run a new case specifically for that finding
3. Capture network traffic for 5 minutes: `sudo tcpdump -i any -w samples/network/incident.pcap -c 500`
4. Analyze both the log and network capture under the new case
5. Run correlation to confirm the connection between sources
6. Generate a PDF report as your documentation

TraceForge v2 — Demo & Usage Guide Last updated: May 2026
github.com/theconsoler/TRACEFORGE-V2